

1. Thread क्या होता है? (Computer Level)

Thread किसी program के अंदर execution की सबसे छोटी इकाई (smallest unit of execution) होती है। जब कोई program चलता है, तो वह कई छोटे-छोटे tasks में बँटकर चलता है। इन tasks को **threads** कहा जाता है।

Structure:

Program → Process → Thread

Definitions:

- **Program:** Disk में पड़ा हुआ code
- **Process:** Running program
- **Thread:** Process के अंदर चलने वाला independent task

👉 एक process में एक या एक से ज्यादा threads हो सकते हैं।

2. CPU में Thread कैसे काम करता है?

CPU instructions execute करता है।

लेकिन CPU बहुत fast होता है, इसलिए वह:

- पहले Thread-1 execute करता है
- फिर Thread-2
- फिर Thread-3

यह switching इतनी तेज होती है कि user को लगता है कि सब कुछ एक साथ चल रहा है।

इस process को **Context Switching** कहते हैं।

👉 Single CPU होने पर भी multithreading possible है।

3. Java Thread क्या होता है?

Java में **Thread** एक class है जो program में multiple tasks को parallel (एक साथ) चलाने की सुविधा देती है।

Java thread का उपयोग:

- Multiple tasks को simultaneously run करने के लिए
- Program को fast और responsive बनाने के लिए
- Games, servers, animations, background tasks में

4. Non-Thread Program (Single Thread Program)

Example:

```
class Test {  
    public static void main(String[] args) {  
  
        for(int i=1; i<=4; i++){  
            System.out.println("Task 1: " + i);  
        }  
  
        for(int i=1; i<=4; i++){  
            System.out.println("Task 2: " + i);  
        }  
    }  
}
```

Output:

```
Task 1: 1  
Task 1: 2  
Task 1: 3  
Task 1: 4  
Task 2: 1  
Task 2: 2  
Task 2: 3  
Task 2: 4
```

Explanation:

- यह **single thread program** है
- Task 1 पूरा होने के बाद ही Task 2 शुरू होगा
- कोई भी काम parallel नहीं हो रहा

5. Java में Thread कैसे बनाते हैं? (Basic Creation)

Thread class को extend करके

```
class MyThread extends Thread {  
    public void run() {  
        System.out.println("Thread is running");  
    }  
}  
  
public class Test {  
    public static void main(String[] args) {  
        MyThread t = new MyThread();  
        t.start();  
    }  
}
```

Important Points:

- run() method में thread का code लिखा जाता है
- start() method नया thread बनाता है
- run() को directly call नहीं करना चाहिए

6. Multithreading Program (Thread वाला Program)

Example: दो threads एक साथ चल रहे हैं

```
class MyThread1 extends Thread {
    public void run() {
        for(int i=1; i<=5; i++){
            System.out.println("Task 1: " + i);
        }
    }
}

class MyThread2 extends Thread {
    public void run() {
        for(int i=1; i<=5; i++){
            System.out.println("Task 2: " + i);
        }
    }
}

public class Test {
    public static void main(String[] args) {

        MyThread1 t1 = new MyThread1();
        MyThread2 t2 = new MyThread2();

        t1.start(); // new thread created
        t2.start(); // another new thread created
    }
}
```

Possible Output:

Task 1: 1
Task 1: 2
Task 2: 1
Task 1: 3
Task 1: 4
Task 2: 2
Task 2: 3

Task 2: 4

Task 2: 5

Task 1: 5

Explanation:

- यहाँ दो threads हैं
- दोनों CPU को share कर रहे हैं
- Output का order fix नहीं होता
- दोनों tasks **simultaneously execute** होते हैं

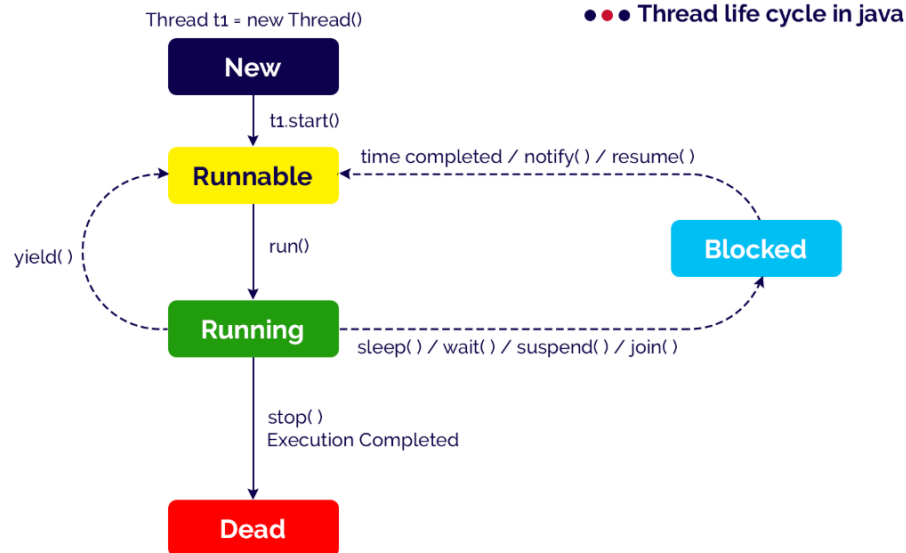
7. Single Thread vs Multithreading

Single Thread	Multithreading
एक ही task एक समय पर	Multiple tasks एक साथ
Slow execution	Fast execution
Blocking problem	Better performance
Simple programs	Games, servers, apps

8. Important Exam Points

- Thread execution start() method से शुरू होता है
- हर thread का अपना stack होता है
- Output order predictable नहीं होता
- Java में multithreading performance बढ़ाती है

Thread Life Cycle States



Thread Life Cycle States (थ्रेड जीवन चक्र की अवस्थाएँ)

एक जावा थ्रेड मुख्य रूप से इन 5 अवस्थाओं में हो सकता है:

1. New (नया थ्रेड)

जब हम Thread क्लास का ऑब्जेक्ट बनाते हैं, तो थ्रेड **New** स्टेट में होता है। इस समय थ्रेड अभी तक सक्रिय नहीं हुआ होता है।

- **Example:** Thread t = new Thread();

2. Runnable (चलाने योग्य)

जब हम start() मेथड को कॉल करते हैं, तो थ्रेड **New** से **Runnable** स्टेट में चला जाता है। यहाँ थ्रेड चलने के लिए तैयार है और थ्रेड शेड्यूलर (Thread Scheduler) के द्वारा चुने जाने का इंतज़ार कर रहा है।

3. Running (चल रहा है)

जब थ्रेड शेड्यूलर थ्रेड को प्रोसेसर समय (CPU time) दे देता है, तो थ्रेड **Running** स्टेट में आ जाता है। यहाँ उसका run() मेथड निष्पादित (execute) हो रहा होता है।

4. Blocked / Waiting (रुका हुआ)

थ्रेड अस्थायी रूप से निष्क्रिय हो जाता है यदि:

- वह किसी दूसरे थ्रेड के रिसोर्स को छोड़ने का इंतज़ार कर रहा हो (**Blocked**)।

- उसे sleep(), wait(), या join() मेथड द्वारा रुकने के लिए कहा गया हो (**Waiting**)।

5. Terminated / Dead (समाप्त)

जब run() मेथड का काम पूरा हो जाता है या थ्रेड में कोई ऐसी एरर आ जाए जिसे संभाला न जा सके, तो थ्रेड **Terminated** स्टेट में चला जाता है। एक बार थ्रेड मर जाने के बाद उसे फिर से शुरू नहीं किया जा सकता।